

Tugas Besar 2
Convolutional Neural Network dan Recurrent Neural Network
IF3270 Pembelajaran Mesin



Kelompok 41: ooooo

Buege Mahara Putra 13523037

Program Studi Teknik Informatika
Institut Teknologi Bandung

2026

Daftar Isi

1 Deskripsi Persoalan	5
1.1 Latar Belakang	5
1.2 Bagian 1: Klasifikasi Gambar dengan CNN	5
1.2.1 Dataset	5
1.2.2 Persoalan dan Tujuan	5
1.2.3 Ruang Lingkup Eksperimen	6
1.3 Bagian 2: Image Captioning dengan RNN dan LSTM	6
1.3.1 Dataset	6
1.3.2 Persoalan dan Tujuan	6
1.3.3 Ruang Lingkup Eksperimen	7
2 Penjelasan Implementasi	8
2.1 Arsitektur Keseluruhan	8
2.2 Layer Interface (src/common/base.py)	8
2.3 Aktivasi (src/common/activations.py)	8
2.4 Layer Umum (src/common/layers.py)	9
2.4.1 Dense	9
2.4.2 Embedding	9
2.5 Layer CNN (src/cnn/layers.py)	9
2.5.1 Helper Function: _sliding_windows	9
2.5.2 Conv2D	10
2.5.3 LocallyConnected2D	10
2.5.4 MaxPooling2D dan AveragePooling2D	10
2.5.5 GlobalAveragePooling2D dan GlobalMaxPooling2D	10
2.5.6 Flatten	11
2.6 Model CNN (src/cnn/model.py)	11
2.7 Cell Recurrent (src/captioning/rnn/layers.py dan src/captioning/lstm/layers.py)	11

2.7.1	SimpleRNNCell	11
2.7.2	LSTMCell	11
2.8	Decoder Captioning (src/captioning/rnn/decoder.py dan src/captioning/lstm/decoder.py)	12
2.9	Preprocessing Data	12
2.9.1	Image (src/cnn/data.py)	12
2.9.2	Caption (src/captioning/text_preprocessing.py)	12
3	Penjelasan Forward Propagation	13
3.1	Forward Propagation CNN	13
3.1.1	Alur Umum	13
3.1.2	Conv2D	13
3.1.3	LocallyConnected2D	13
3.1.4	Pooling	14
3.1.5	Dense Output	14
3.2	Forward Propagation SimpleRNN	14
3.2.1	Arsitektur Decoder Pre-inject	14
3.2.2	SimpleRNN Cell pada Satu Timestep	14
3.3	Forward Propagation LSTM	15
3.3.1	LSTM Cell pada Satu Timestep	15
3.3.2	Arsitektur Decoder LSTM Pre-inject	15
4	Hasil Pengujian	16
4.1	CNN	16
4.1.1	Perbandingan Shared vs Non-Shared Parameter	16
4.1.2	Pengaruh Jumlah Layer Konvolusi	17
4.1.3	Pengaruh Banyak Filter per Layer Konvolusi	18
4.1.4	Pengaruh Ukuran Filter per Layer Konvolusi	19
4.1.5	Pengaruh Jenis Pooling Layer	20
4.1.6	Perbandingan Keras vs From Scratch	21

4.2	SimpleRNN dan LSTM untuk Image Captioning	22
4.2.1	Perbandingan Jumlah Layer: RNN	22
4.2.2	Perbandingan Jumlah Layer: LSTM	23
4.2.3	Perbandingan Ukuran Hidden State: RNN	24
4.2.4	Perbandingan Ukuran Hidden State: LSTM	24
4.2.5	Perbandingan RNN vs LSTM	25
4.2.6	Perbandingan Keras vs From Scratch	25
4.2.7	Pengaruh Panjang Maksimum Caption terhadap BLEU-4	26
5	Kesimpulan	28
6	Pembagian Tugas	30
	Referensi	31
A	Lampiran: Penggunaan AI	32

1 Deskripsi Persoalan

1.1 Latar Belakang

Tugas besar ini mencakup dua bagian pembelajaran mesin yang saling melengkapi, yaitu klasifikasi gambar menggunakan Convolutional Neural Network (CNN) dan pembuatan caption gambar (*image captioning*) menggunakan arsitektur recurrent berbasis SimpleRNN dan LSTM. Kedua bagian diselesaikan dengan strategi dua tahap. Model dilatih menggunakan Keras, kemudian bobot yang telah dilatih dimuat ke dalam implementasi *forward propagation* berbasis NumPy yang ditulis dari nol. Pendekatan ini memungkinkan verifikasi bahwa implementasi lokal menghasilkan keluaran yang konsisten dengan kerangka kerja standar.

1.2 Bagian 1: Klasifikasi Gambar dengan CNN

1.2.1 Dataset

Dataset yang digunakan adalah Intel Image Classification, yang terdiri dari gambar pemandangan alam dan perkotaan berwarna berukuran 150×150 piksel. Dataset ini dibagi menjadi tiga bagian: *train*, *validation*, dan *test*. Terdapat enam kelas label:

Indeks	Nama Kelas	Deskripsi
0	buildings	Bangunan dan struktur buatan manusia
1	forest	Hutan dan vegetasi lebat
2	glacier	Gletser dan lanskap bersalju
3	mountain	Pegunungan
4	sea	Lautan dan perairan
5	street	Jalanan perkotaan

1.2.2 Persoalan dan Tujuan

Persoalan yang diselesaikan adalah mengklasifikasikan setiap gambar ke salah satu dari enam kelas di atas. Model CNN dilatih dengan fungsi loss *sparse categorical cross-entropy* dan optimizer Adam. Metrik utama yang digunakan adalah macro F1-score pada split test.

1.2.3 Ruang Lingkup Eksperimen

Dilakukan 16 variasi model CNN berbasis Conv2D dengan parameter bersama (*shared*), dikombinasikan dari empat dimensi variasi:

- **Jumlah layer konvolusi:** 2 variasi
- **Jumlah filter:** 2 variasi
- **Ukuran kernel:** 2 variasi
- **Jenis pooling:** max pooling atau average pooling

Selain itu, dilakukan perbandingan antara model Conv2D (*shared parameter*) dengan model LocallyConnected2D (*non-shared parameter*) pada konfigurasi terbaik, untuk menganalisis pengaruh pembagian bobot terhadap akurasi, jumlah parameter, dan efisiensi komputasi.

1.3 Bagian 2: Image Captioning dengan RNN dan LSTM

1.3.1 Dataset

Dataset yang digunakan adalah Flickr8k, yang terdiri dari 8.092 gambar dengan masing-masing 5 caption deskriptif dalam bahasa Inggris. Dataset dibagi menjadi tiga split.

Split	Jumlah Gambar	Jumlah Caption
Train	6.000	30.000
Validation	1.000	5.000
Test	1.000	5.000
Total	8.092	40.460

1.3.2 Persoalan dan Tujuan

Persoalan yang diselesaikan adalah menghasilkan caption teks secara otomatis untuk gambar yang diberikan. Arsitektur yang digunakan adalah *encoder-decoder* dengan metode injeksi pre-inject. Fitur gambar yang diekstrak dari CNN pretrained diproyeksikan ke ruang embedding dan dimasukkan sebagai *input timestep pertama* pada decoder recurrent.

Encoder gambar adalah model InceptionV3 pretrained ImageNet tanpa *classification head*, dibekukan selama training (*frozen*). Decoder dilatih dengan strategi *teacher forcing*. Evaluasi menggunakan metrik BLEU-4 dan METEOR.

1.3.3 Ruang Lingkup Eksperimen

Dilakukan 12 variasi model decoder yang terbagi dua:

- **6 model SimpleRNN:** 3 variasi jumlah *recurrent layer* $\times$ 2 variasi *hidden size*
- **6 model LSTM:** variasi yang sama

Analisis mencakup pengaruh jumlah layer recurrent, pengaruh *hidden size*, perbandingan SimpleRNN vs LSTM, serta variasi panjang maksimum caption terhadap BLEU-4 pada arsitektur terbaik.

2 Penjelasan Implementasi

2.1 Arsitektur Keseluruhan

Implementasi diorganisasi dalam folder `src/` yang terbagi menjadi empat modul utama:

Modul	Isi
<code>src/common/</code>	Layer dasar, aktivasi, metrik, serialisasi, plotting
<code>src/cnn/</code>	Layer CNN, model, data loader, Keras factory, evaluasi
<code>src/captioning/rnn/</code>	SimpleRNN cell, decoder NumPy, Keras decoder factory
<code>src/captioning/lstm/</code>	LSTM cell, decoder NumPy, Keras decoder factory

Seluruh eksperimen, training, evaluasi, dan ekspor artefak dilakukan di notebook (`notebooks/cnn.ipynb` dan `notebooks/captioning.ipynb`). Modul `src/` hanya berisi komponen reusable.

2.2 Layer Interface (`src/common/base.py`)

Semua layer implementasi lokal merupakan subclass dari kelas `Layer`. Interface yang wajib diimplementasikan oleh setiap layer adalah:

- `build(input_shape)`: inisialisasi bobot berdasarkan dimensi input
- `forward(inputs)`: komputasi *forward pass*
- `__call__(inputs)`: memanggil `build` jika belum dibangun, lalu `forward`
- `get_weights()`: mengembalikan daftar array bobot
- `set_weights(weights)`: memuat bobot dari luar (termasuk dari Keras)
- `compute_output_shape(input_shape)`: menghitung shape keluaran tanpa komputasi aktual
- `get_config()`: mengembalikan konfigurasi layer sebagai dictionary
- `parameter_count()`: mengembalikan jumlah total parameter

Kelas `Sequential` memungkinkan komposisi layer secara berurutan, mirip dengan `keras.Sequential`. Kelas `Model` digunakan untuk arsitektur yang lebih kompleks seperti decoder.

2.3 Aktivasi (`src/common/activations.py`)

Semua fungsi aktivasi diimplementasikan dengan NumPy.

Fungsi	Implementasi
relu	$\max(0, x)$
sigmoid	$1/(1 + e^{-x})$
tanh	$\text{np.tanh}(x)$
softmax	$e^{x_i} / \sum_j e^{x_j}$
linear	identitas

2.4 Layer Umum (src/common/layers.py)

2.4.1 Dense

Layer Dense mengimplementasikan transformasi linear dengan aktivasi opsional:

$$\mathbf{y} = \mathbf{x} \cdot \mathbf{W} + \mathbf{b} \quad (1)$$

di mana $\mathbf{W}$ adalah bobot kernel dan $\mathbf{b}$ adalah bias. Bobot dimuat dari layer Keras dengan format `[kernel, bias]`.

2.4.2 Embedding

Layer Embedding mengubah indeks token integer menjadi vektor representasi:

$$\mathbf{e}_t = E[t], \quad E \in \mathbb{R}^{V \times d_e} \quad (2)$$

di mana V adalah ukuran vocabulary dan d_e adalah dimensi embedding. Operasi forward hanya melakukan pengindeksan baris (`embeddings[token_indices]`).

2.5 Layer CNN (src/cnn/layers.py)

2.5.1 Helper Function: `_sliding_windows`

Operasi inti yang digunakan oleh semua layer konvolusi dan pooling adalah ekstraksi *sliding windows*. Fungsi `_sliding_windows` menerima tensor input berformat *channel-last* (N, H, W, C) dan menghasilkan tensor patch $(N, H_{\text{out}}, W_{\text{out}}, k_h, k_w, C)$, di mana H_{out} dan W_{out} adalah dimensi spasial keluaran.

2.5.2 Conv2D

Layer Conv2D mengimplementasikan konvolusi 2D dengan bobot bersama (*shared parameter*). Bobot kernel memiliki shape (k_h, k_w, C_{in}, F) , di mana F adalah jumlah filter. Operasi forward menggunakan `np.tensordot`.

```
1 windows = _sliding_windows(inputs, kernel_size, strides, padding)
2 outputs = np.tensordot(windows, kernel,
3                        axes=([3, 4, 5], [0, 1, 2]))
4 outputs = outputs + bias
5 return activation(outputs)
```

Hasilnya memiliki shape (N, H_{out}, W_{out}, F) .

2.5.3 LocallyConnected2D

Layer LocallyConnected2D adalah varian konvolusi dengan bobot *non-shared*: setiap posisi keluaran memiliki kernel tersendiri. Bobot kernel memiliki shape $(H_{out}, W_{out}, k_h, k_w, C_{in}, F)$. Operasi forward menggunakan `np.einsum`.

```
1 windows = _sliding_windows(inputs, kernel_size, strides, padding)
2 outputs = np.einsum("nxyhwc,xyhwcf->nxyf", windows, kernel)
3 outputs = outputs + bias
4 return activation(outputs)
```

Akibat bobot per posisi, jumlah parameter jauh lebih besar dibanding Conv2D dengan konfigurasi yang sama.

2.5.4 MaxPooling2D dan AveragePooling2D

Kedua layer pooling menggunakan `_sliding_windows` untuk mengekstrak region, kemudian mereduksi dimensi spasial:

- MaxPooling2D: `np.max(windows, axis=(3, 4))`
- AveragePooling2D: `np.mean(windows, axis=(3, 4))`
dengan koreksi jumlah elemen valid untuk padding "same"

2.5.5 GlobalAveragePooling2D dan GlobalMaxPooling2D

Mereduksi seluruh dimensi spasial menjadi satu nilai per channel:

- GlobalAveragePooling2D: `np.mean(array, axis=(1, 2))`
- GlobalMaxPooling2D: `np.max(array, axis=(1, 2))`

Menghasilkan output shape (N, C) .

2.5.6 Flatten

Mengubah tensor (N, H, W, C) menjadi $(N, H \cdot W \cdot C)$:

```
1 return array.reshape((array.shape[0], -1))
```

2.6 Model CNN (src/cnn/model.py)

Kelas `CNNModel` adalah subclass `Sequential` yang menyusun layer-layer CNN dan menyediakan method:

- `predict_proba(inputs)`: mengembalikan probabilitas softmax (N, K)
- `predict(inputs)`: mengembalikan indeks kelas via `argmax`
- `forward_single(image)`: menangani gambar tunggal (H, W, C) dengan menambahkan dimensi batch
- `load_keras_weights(keras_model)`: memuat bobot layer demi layer dari model Keras

2.7 Cell Recurrent (src/captioning/rnn/layers.py dan src/captioning/lstm/layers.py)

2.7.1 SimpleRNNCell

Kelas `SimpleRNNCell` adalah subclass `RecurrentCell` yang mengimplementasikan satu timestep SimpleRNN. Bobot dimuat dari Keras dalam format `[kernel, recurrent_kernel, bias]`.

Method `step(inputs, state)` menerima input (N, d_{in}) dan hidden state (N, H) , kemudian mengembalikan output dan state baru (N, H) .

2.7.2 LSTMCell

Kelas `LSTMCell` mengimplementasikan satu timestep LSTM. Bobot dimuat dari Keras dalam format `[kernel, recurrent_kernel, bias]` dengan dimensi $4H$ untuk

mendukung empat gate. State terdiri dari pasangan (h, c) .

Method `step(inputs, state)` menerima input (N, d_{in}) dan state (h_{prev}, c_{prev}) , kemudian mengembalikan output h_t dan state baru (h_t, c_t) .

2.8 Decoder Captioning (`src/captioning/rnn/decoder.py` dan `src/captioning/lstm/decoder.py`)

Kedua decoder mengimplementasikan strategi pre-inject. Fitur gambar diproyeksikan ke ruang embedding, kemudian digunakan sebagai input pada timestep pertama untuk memperbarui hidden state sebelum pemrosesan token caption dimulai.

Komponen decoder adalah:

1. `image_projection`: Dense $(F \rightarrow d_e)$
2. `embedding`: Embedding $(V \times d_e)$
3. `cells`: daftar SimpleRNNCell atau LSTMCell yang bertumpuk
4. `output_dense`: Dense $(H \rightarrow V)$ dengan aktivasi softmax

Method `forward_sequence(image_features, token_ids)` menjalankan seluruh sekuens dengan teacher forcing dan mengembalikan distribusi probabilitas (N, T, V) .

2.9 Preprocessing Data

2.9.1 Image (`src/cnn/data.py`)

Image dimuat menggunakan `PIL.Image.open`, di-resize ke ukuran target, dikonversi ke array NumPy, dan dinormalisasi ke rentang $[0, 1]$. Augmentasi opsional mencakup flip horizontal, rotasi ($\pm 15^\circ$), zoom ($\pm 10\%$), dan translasi ($\pm 8\%$).

2.9.2 Caption (`src/captioning/text_preprocessing.py`)

Preprocessing caption meliputi lowercase, penghapusan tanda baca dengan regex, tokenisasi dengan `str.split()`, pembuatan vocabulary dengan frekuensi minimum, penambahan token khusus (`<pad>`, `<start>`, `<end>`), padding ke panjang seragam, dan pembuatan pasangan input-target untuk teacher forcing.

3 Penjelasan Forward Propagation

3.1 Forward Propagation CNN

3.1.1 Alur Umum

Input gambar dengan format *channel-last* (N, H, W, C) melewati rangkaian layer secara berurutan. Satu atau lebih pasangan layer konvolusi dan pooling, diikuti layer pereduksi dimensi (Flatten atau Global Pooling), lalu layer Dense dengan aktivasi softmax yang menghasilkan distribusi probabilitas atas kelas.

3.1.2 Conv2D

Operasi dasar Conv2D adalah menggeser sebuah kernel kecil berukuran $k_h \times k_w$ di seluruh area spasial input. Pada setiap posisi, dilakukan perkalian elemen per elemen antara patch input dengan kernel, lalu dijumlahkan. Hasilnya adalah satu nilai skalar per filter. Karena kernel yang sama dipakai di semua posisi, Conv2D disebut *shared parameter*.

Implementasinya mengekstrak semua patch sekaligus ke dalam satu tensor besar menggunakan fungsi `_sliding_windows`, lalu menghitung seluruh konvolusi sekaligus dengan `np.tensordot`. Hasil konvolusi ditambah bias, lalu dilewatkan ke fungsi aktivasi.

Terdapat dua jenis padding. Padding "valid" tidak menambahkan piksel di tepi sehingga dimensi keluaran lebih kecil dari input. Padding "same" menambahkan nol secara simetris di sekeliling input agar dimensi keluaran tetap sebanding dengan input.

3.1.3 LocallyConnected2D

LocallyConnected2D bekerja dengan cara yang sama dengan Conv2D, perbedaannya setiap posisi keluaran memiliki kernel tersendiri. Kernel tidak dibagi antar posisi, sehingga model dapat mempelajari pola yang berbeda di setiap lokasi spasial. Konsekuensinya, jumlah parameter jauh lebih besar dibanding Conv2D dengan konfigurasi yang sama.

Implementasinya menggunakan `np.einsum` dengan notasi "`nxyhwc,xyhwcf->nxyf`" untuk menerapkan kernel yang berbeda per posisi secara efisien. Untuk setiap sampel n , posisi output (x, y) , dan filter f , hasil kali patch input (h, w, c) dengan kernel di posisi (x, y) yang bersesuaian dijumlahkan. Sumbu h, w , dan c tidak dimunculkan di sisi kanan sehingga ketiganya direduksi, dan tensor output yang dihasilkan bershape (n, x, y, f) .

3.1.4 Pooling

Layer pooling mereduksi dimensi spasial dengan meringkas setiap region kecil menjadi satu nilai. MaxPooling2D mengambil nilai maksimum dari setiap region, sementara AveragePooling2D mengambil rata-ratanya. GlobalAveragePooling2D dan GlobalMaxPooling2D mereduksi seluruh area spasial menjadi satu nilai per channel, menghasilkan vektor fitur satu dimensi per sampel. Flatten melakukan hal serupa dengan cara meratakan seluruh dimensi non-batch menjadi satu vektor panjang.

3.1.5 Dense Output

Setelah dimensi spasial direduksi, fitur melewati layer Dense. Setiap neuron menghitung kombinasi linear dari seluruh input, ditambah bias. Pada layer output, aktivasi softmax mengubah nilai-nilai tersebut menjadi distribusi probabilitas. Kelas prediksi dipilih sebagai kelas dengan probabilitas tertinggi.

3.2 Forward Propagation SimpleRNN

3.2.1 Arsitektur Decoder Pre-inject

Decoder SimpleRNN menggunakan strategi pre-inject. Sebelum memproses token caption apapun, fitur gambar terlebih dahulu diproyeksikan ke dimensi embedding menggunakan layer Dense, lalu diumpankan ke semua cell recurrent untuk memperbarui hidden state awal. Dengan cara ini, informasi gambar "diinjeksikan" ke dalam state sebelum decoding dimulai.

Setelah injeksi, decoder memproses token caption satu per satu. Setiap token diubah menjadi vektor embedding, diproses oleh semua layer cell recurrent secara berurutan, lalu dilewatkan ke layer Dense output dengan softmax untuk menghasilkan distribusi probabilitas atas vocabulary. Jika terdapat lebih dari satu layer recurrent, output layer ke- l menjadi input layer ke- $l + 1$.

3.2.2 SimpleRNN Cell pada Satu Timestep

Pada setiap timestep, SimpleRNN cell menerima input saat ini dan hidden state dari timestep sebelumnya, lalu menghasilkan hidden state baru:

$$h_t = \tanh(x_t \cdot W_{\text{in}} + h_{t-1} \cdot W_{\text{rec}} + b) \quad (3)$$

W_{in} adalah bobot untuk input saat ini, W_{rec} adalah bobot recurrent yang menghubungkan state antar timestep, dan b adalah bias. Hidden state baru h_t sekaligus menjadi output cell pada timestep tersebut.

3.3 Forward Propagation LSTM

3.3.1 LSTM Cell pada Satu Timestep

LSTM memiliki dua state, yaitu hidden state h seperti pada SimpleRNN, dan cell state c . Cell state berfungsi sebagai memori jangka panjang yang dapat membawa informasi melewati banyak timestep.

Pada setiap timestep, LSTM menghitung empat nilai gate dari kombinasi linear input saat ini dan hidden state sebelumnya. Keempat gate tersebut adalah:

- **Input gate** i : menentukan seberapa banyak informasi baru yang masuk ke cell state
- **Forget gate** f : menentukan seberapa banyak informasi lama yang dipertahankan dari cell state sebelumnya
- **Cell candidate** g : nilai baru yang akan ditambahkan ke cell state
- **Output gate** o : menentukan bagian cell state yang akan dikeluarkan sebagai hidden state

Cell state diperbarui dengan melupakan sebagian informasi lama (dikali f) lalu menambahkan informasi baru (dikali i):

$$c_t = f \odot c_{t-1} + i \odot g \quad (4)$$

$$h_t = o \odot \tanh(c_t) \quad (5)$$

Bobot LSTM memiliki ukuran empat kali lebih besar dari SimpleRNN karena harus menghitung keempat gate secara bersamaan.

3.3.2 Arsitektur Decoder LSTM Pre-inject

Alur decoder LSTM identik dengan SimpleRNN. Perbedaannya hanya pada state yang terdiri dari pasangan (h, c) per layer, dan keduanya diinisialisasi dengan nol.

4 Hasil Pengujian

4.1 CNN

Seluruh 16 model Conv2D dilatih pada dataset Intel Image Classification dengan input 150×150 piksel, menggunakan optimizer Adam, dropout 0.5, dan dense layer 128 unit. Hasil macro F1 pada split test dirangkum pada Tabel 1.

Tabel 1 Hasil macro F1 test seluruh 16 variasi model Conv2D

Model	Layer	Filter	Kernel	Pooling	F1 Test
L4_F32-64-128-256_K3_Pm	4	32 - 64 - 128 - 256	3	max	0.8794
L4_F32-64-128-256_K5_Pm	4	32 - 64 - 128 - 256	5	max	0.8731
L4_F64-128-256-512_K3_Pa	4	64 - 128 - 256 - 512	3	average	0.8723
L4_F64-128-256-512_K3_Pm	4	64 - 128 - 256 - 512	3	max	0.8720
L4_F32-64-128-256_K5_Pa	4	32 - 64 - 128 - 256	5	average	0.8706
L4_F64-128-256-512_K5_Pa	4	64 - 128 - 256 - 512	5	average	0.8691
L3_F64-128-256_K5_Pa	3	64 - 128 - 256	5	average	0.8661
L3_F32-64-128_K5_Pm	3	32 - 64 - 128	5	max	0.8649
L4_F64-128-256-512_K5_Pm	4	64 - 128 - 256 - 512	5	max	0.8637
L3_F64-128-256_K5_Pm	3	64 - 128 - 256	5	max	0.8588
L3_F32-64-128_K5_Pa	3	32 - 64 - 128	5	average	0.8566
L3_F64-128-256_K3_Pa	3	64 - 128 - 256	3	average	0.8540
L4_F32-64-128-256_K3_Pa	4	32 - 64 - 128 - 256	3	average	0.8481
L3_F32-64-128_K3_Pm	3	32 - 64 - 128	3	max	0.8452
L3_F32-64-128_K3_Pa	3	32 - 64 - 128	3	average	0.8452
L3_F64-128-256_K3_Pm	3	64 - 128 - 256	3	max	0.8284

Model terbaik berdasarkan validation F1 adalah conv_L4_F64-128-256-512_K3_Pa dengan val F1 = 0.8867 dan test F1 = 0.8723. Model ini memiliki 4 layer konvolusi, filter schedule 64-128-256-512, kernel 3×3 , dan average pooling, dengan total 6.860.294 parameter.

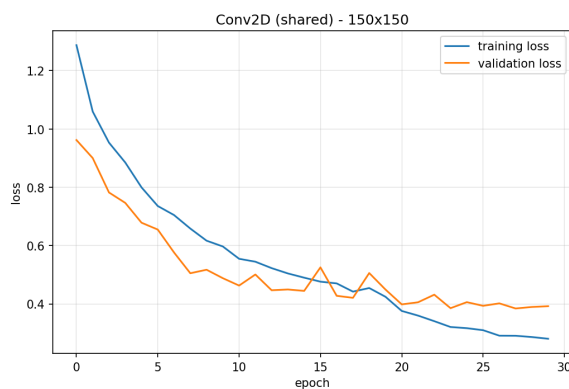
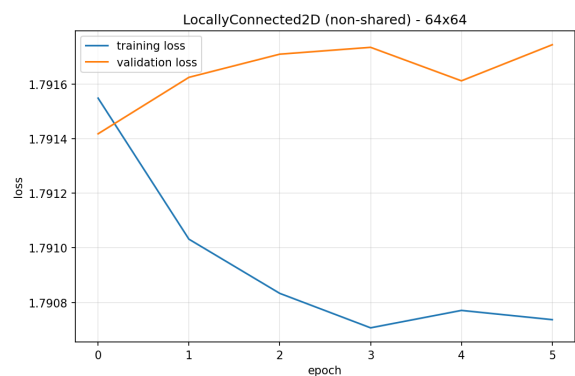
4.1.1 Perbandingan Shared vs Non-Shared Parameter

Model LocallyConnected2D (non-shared) dikonfigurasi dengan arsitektur yang sama dengan model terbaik Conv2D, namun dilatih pada input 64×64 piksel karena keterbatasan memori akibat ledakan jumlah parameter. Perbandingan keduanya ditampilkan pada Tabel 2.

Tabel 2 Perbandingan Conv2D (shared) dan LocallyConnected2D (non-shared)

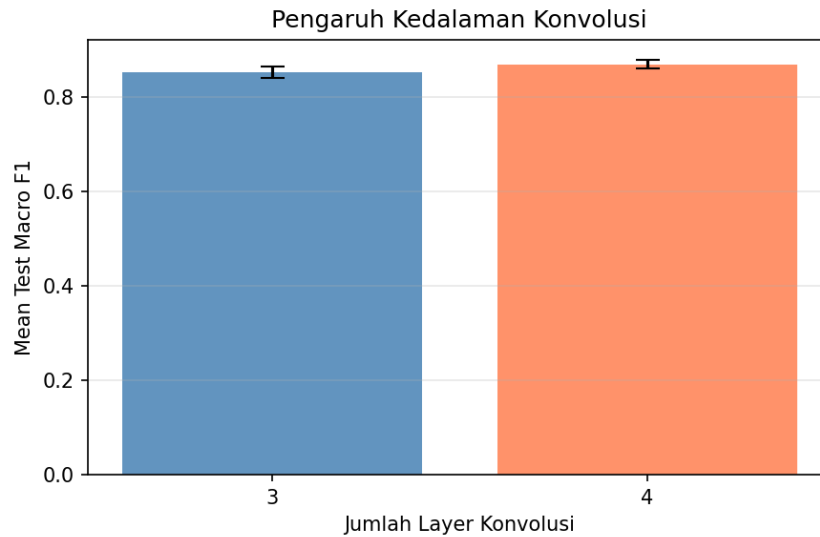
Arsitektur	Input	Implementasi	Parameter	F1 Test
Conv2D (shared)	150×150	Keras	6.860.294	0.8723
LocallyConnected2D (non-shared)	64×64	Keras	235.111.302	0.0496

LocallyConnected2D memiliki parameter 34 kali lebih banyak (235 juta vs 6,8 juta), namun menghasilkan F1 yang sangat rendah (0.0496, hampir setara tebakan acak untuk 6 kelas). Hal ini mengindikasikan bahwa model non-shared gagal konvergen, kemungkinan karena ruang parameter yang terlalu besar sehingga data training tidak cukup untuk mengoptimalkan setiap kernel secara independen. Conv2D dengan parameter shared jauh lebih efisien dan efektif.

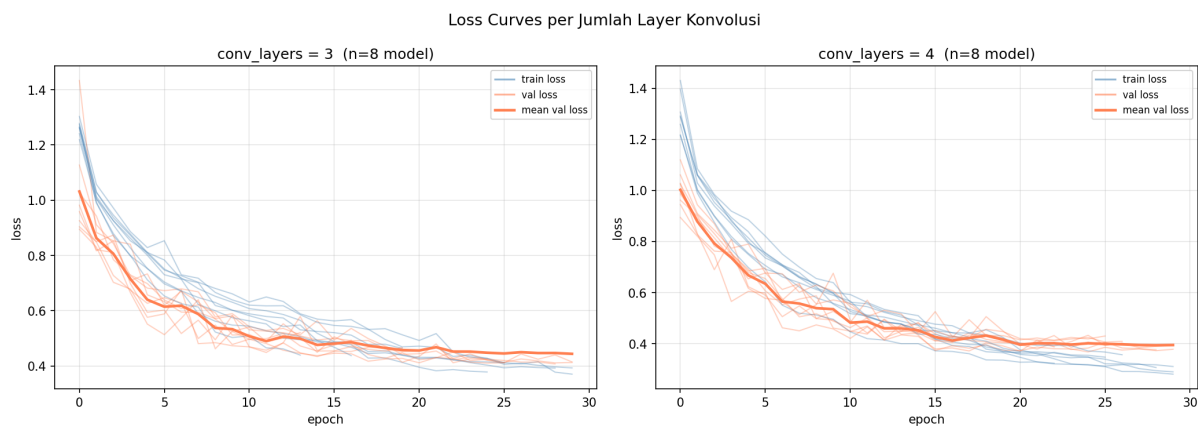
**(a)** Loss model Conv2D terbaik**(b)** Loss model LocallyConnected2D**Gambar 1** Perbandingan kurva loss Conv2D (shared) vs LocallyConnected2D (non-shared)

4.1.2 Pengaruh Jumlah Layer Konvolusi

Model dengan 4 layer konvolusi secara konsisten mengungguli model dengan 3 layer. Rata-rata test F1 model 4-layer adalah 0.8685, sedangkan model 3-layer adalah 0.8524. Layer konvolusi yang lebih banyak memungkinkan model mempelajari representasi fitur yang lebih hierarkis dan abstrak.



(a) Macro F1 per jumlah layer

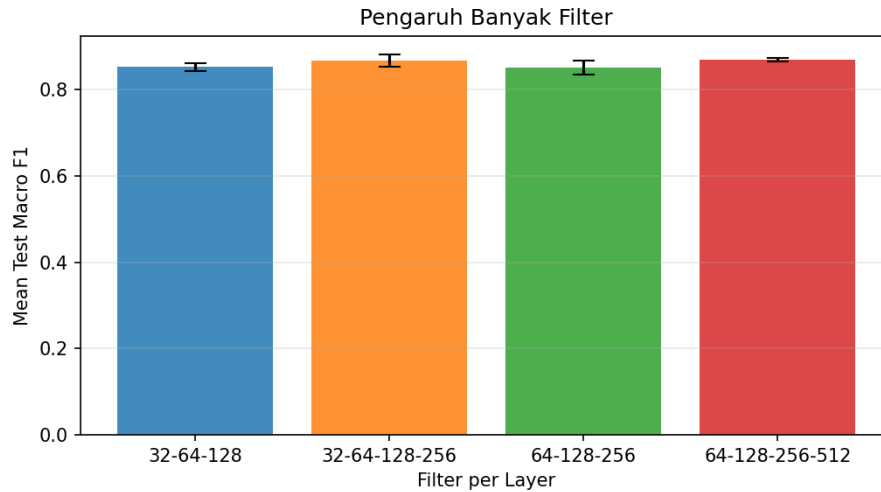


(b) Kurva loss per jumlah layer

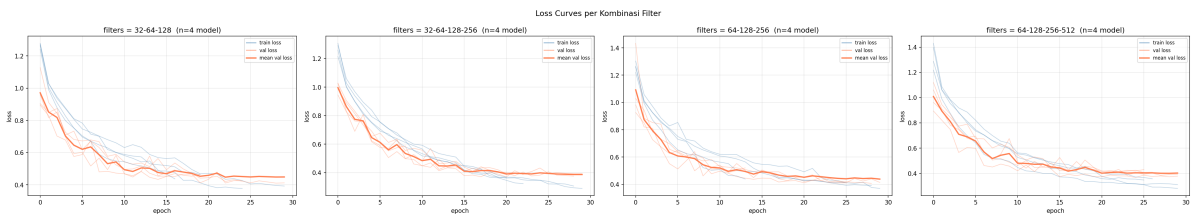
Gambar 2 Pengaruh jumlah layer konvolusi terhadap performa

4.1.3 Pengaruh Banyak Filter per Layer Konvolusi

Dua variasi filter yang diuji adalah schedule kecil (32-64-128 untuk 3 layer, 32-64-128-256 untuk 4 layer) dan besar (64-128-256 untuk 3 layer, 64-128-256-512 untuk 4 layer). Rata-rata F1 keduanya hampir setara, 0.8604 untuk filter kecil dan 0.8606 untuk filter besar. Jumlah filter yang lebih besar tidak memberikan keunggulan yang signifikan, namun meningkatkan jumlah parameter secara substansial sehingga membutuhkan waktu training lebih lama.



(a) Macro F1 per schedule filter

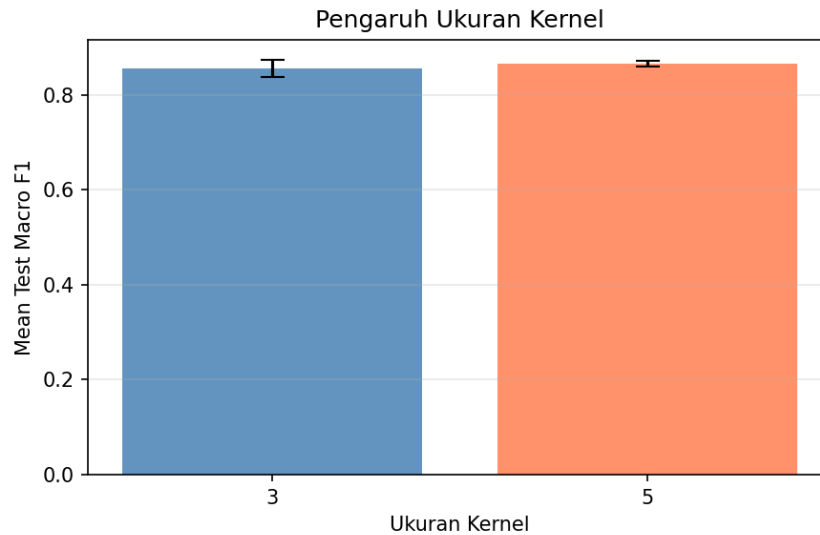


(b) Kurva loss per schedule filter

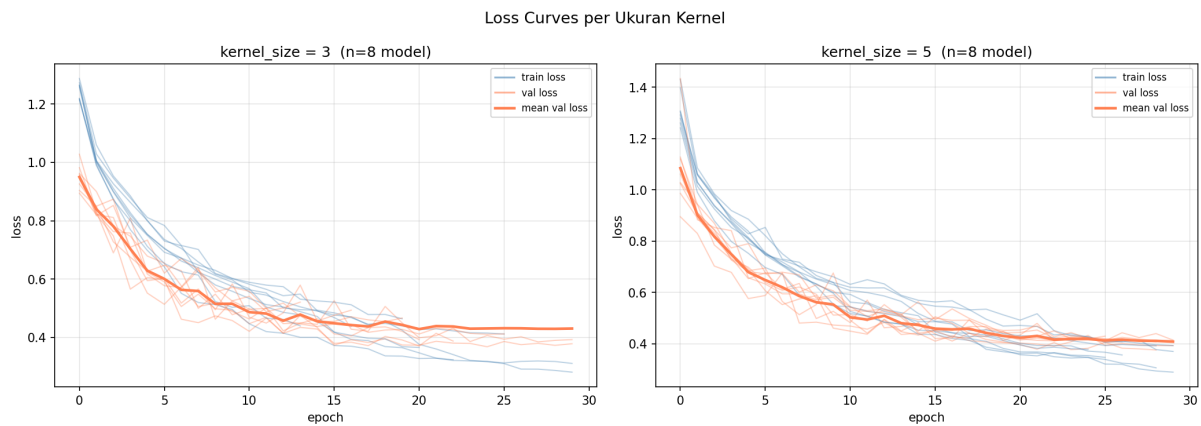
Gambar 3 Pengaruh jumlah filter per layer konvolusi terhadap performa

4.1.4 Pengaruh Ukuran Filter per Layer Konvolusi

Kernel 5×5 menghasilkan rata-rata test F1 = 0.8654, sedikit lebih tinggi dibanding kernel 3×3 dengan rata-rata 0.8556. Kernel yang lebih besar menangkap konteks spasial yang lebih luas pada setiap operasi konvolusi, yang tampaknya bermanfaat pada dataset ini. Namun model terbaik keseluruhan justru menggunakan kernel 3×3 , sehingga ukuran kernel bukan faktor yang berdiri sendiri.



(a) Macro F1 per ukuran kernel

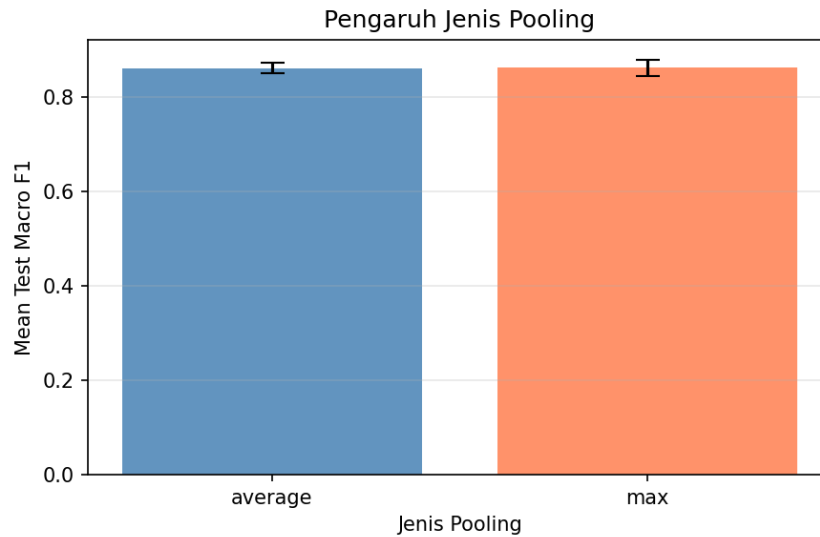


(b) Kurva loss per ukuran kernel

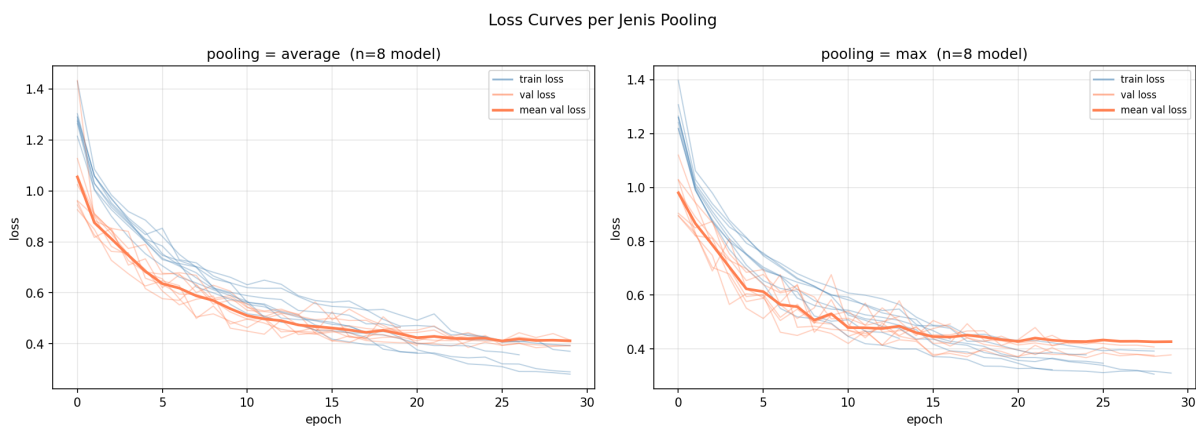
Gambar 4 Pengaruh ukuran kernel terhadap performa

4.1.5 Pengaruh Jenis Pooling Layer

Max pooling dan average pooling menghasilkan performa yang sangat serupa, rata-rata F1 max pooling = 0.8607, average pooling = 0.8603. Perbedaan ini tidak signifikan secara praktis. Namun model dengan F1 tertinggi (0.8794) menggunakan max pooling, sedangkan model terbaik berdasarkan val F1 menggunakan average pooling.



(a) Macro F1 per jenis pooling



(b) Kurva loss per jenis pooling

Gambar 5 Pengaruh jenis pooling layer terhadap performa

4.1.6 Perbandingan Keras vs From Scratch

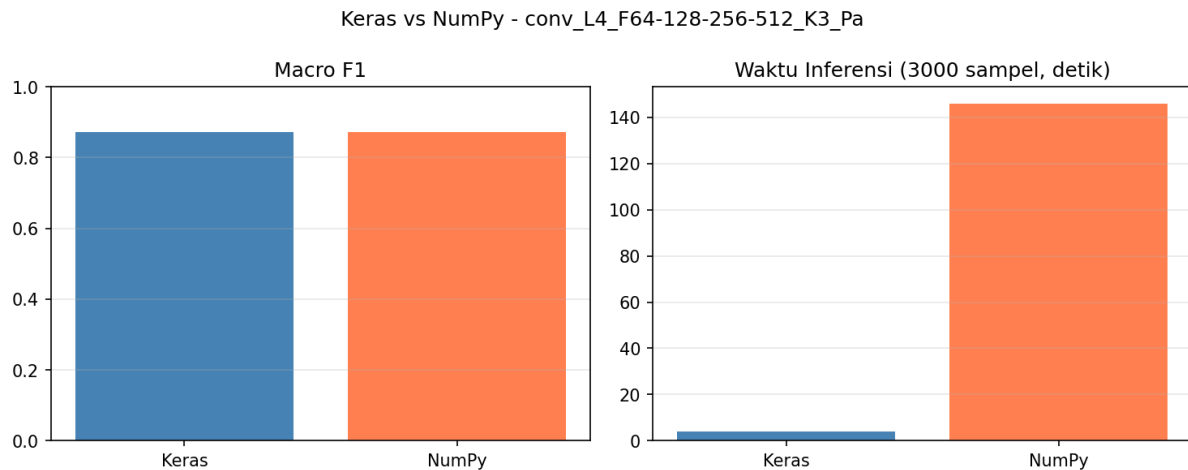
Implementasi forward propagation NumPy diuji pada model terbaik menggunakan 3.000 sampel test. Model yang diuji adalah conv_L4_F64-128-256-512_K3_Pa. Hasilnya ditampilkan pada Tabel 3.

Tabel 3 Perbandingan Keras vs implementasi NumPy pada model terbaik

Implementasi	Macro F1	Waktu (s)	Agreement
Keras	0.8723	3.91	100%
NumPy	0.8723	145.94	

Kedua implementasi menghasilkan prediksi yang identik pada seluruh 3.000 sampel

test (agreement 100%), memvalidasi kebenaran implementasi forward propagation NumPy. Waktu inferensi NumPy sekitar 37 kali lebih lambat dari Keras karena Keras memanfaatkan operasi GPU yang dioptimalkan, sedangkan implementasi NumPy berjalan secara sekuensial di CPU.



Gambar 6 Perbandingan prediksi Keras vs implementasi NumPy

4.2 SimpleRNN dan LSTM untuk Image Captioning

4.2.1 Perbandingan Jumlah Layer: RNN

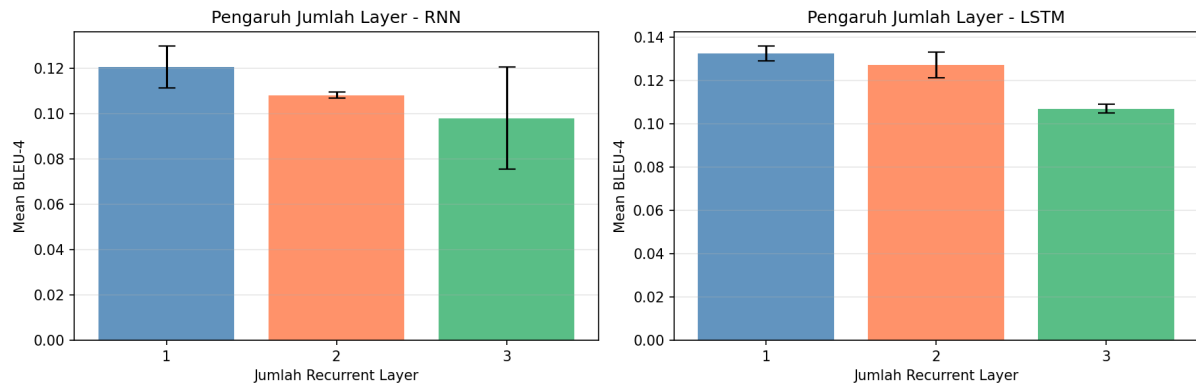
Hasil BLEU-4 dan METEOR enam model SimpleRNN pada split test ditampilkan pada Tabel 4.

Tabel 4 Hasil 6 model SimpleRNN diurutkan per jumlah layer

Model	Layer	Hidden	BLEU-4	METEOR	Parameter
rnn_L1_H256	1	256	0.1142	0.3234	2.986.431
rnn_L1_H512	1	512	0.1272	0.3352	4.411.839
rnn_L2_H256	2	256	0.1073	0.3142	3.117.759
rnn_L2_H512	2	512	0.1092	0.3097	4.936.639
rnn_L3_H256	3	256	0.0823	0.2994	3.249.087
rnn_L3_H512	3	512	0.1140	0.3120	5.461.439

Model satu layer menghasilkan rata-rata BLEU-4 tertinggi (0.1207), diikuti dua layer (0.1083) dan tiga layer (0.0982). Penurunan sebesar 18.6% dari L1 ke L3 mengindikasikan bahwa model yang lebih dalam sulit dioptimalkan pada dataset berukuran 6.000 gambar. Varian terbesar pada L3 (std = 0.0159) menunjukkan sensitivitas yang lebih tinggi terhadap

pilihan hidden size. Waktu decoding juga meningkat hampir linear, 4.8 s (L1), 6.5 s (L2), dan 8.3 s (L3) per 1.000 gambar.



Gambar 7 Pengaruh jumlah recurrent layer terhadap rata-rata BLEU-4 (SimpleRNN dan LSTM)

4.2.2 Perbandingan Jumlah Layer: LSTM

Hasil enam model LSTM ditampilkan pada Tabel 5.

Tabel 5 Hasil 6 model LSTM diurutkan per jumlah layer

Model	Layer	Hidden	BLEU-4	METEOR	Parameter
lstm_L1_H256	1	256	0.1301	0.3438	3.380.415
lstm_L1_H512	1	512	0.1349	0.3430	5.593.023
lstm_L2_H256	2	256	0.1315	0.3384	3.905.727
lstm_L2_H512	2	512	0.1231	0.3380	7.692.223
lstm_L3_H256	3	256	0.1084	0.3209	4.431.039
lstm_L3_H512	3	512	0.1057	0.3171	9.791.423

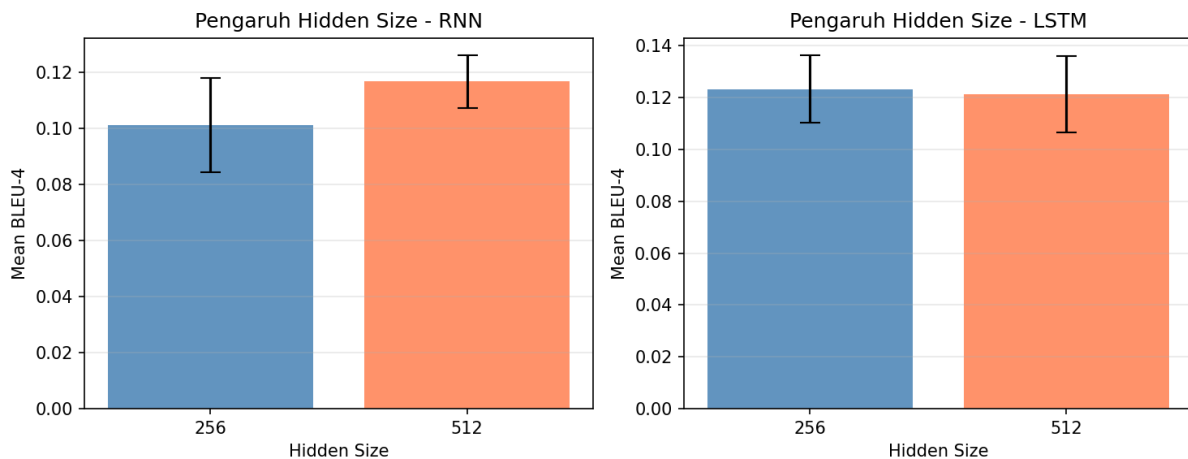
Pola yang sama dengan SimpleRNN teramati. Model satu layer unggul dengan rata-rata BLEU-4 0.1325, diikuti dua layer (0.1273) dan tiga layer (0.1071). Penurunan dari L1 ke L3 sebesar 19.2%. Varian antar konfigurasi pada LSTM lebih kecil dibanding SimpleRNN, menandakan stabilitas training yang lebih baik berkat mekanisme gating. Waktu decoding per 1.000 gambar: 9.4 s (L1), 16.3 s (L2), dan 24.3 s (L3). Setiap tambahan layer LSTM menambah komputasi empat gate per timestep, sehingga biaya per layer lebih tinggi dibanding SimpleRNN.

4.2.3 Perbandingan Ukuran Hidden State: RNN

Tabel 6 Rata-rata BLEU-4 SimpleRNN per hidden size

Hidden	Mean BLEU-4	Std	Mean Decode Time (s)
256	0.1013	0.0134	4.8
512	0.1168	0.0076	8.2

Pada SimpleRNN, hidden size 512 memberikan BLEU-4 rata-rata lebih tinggi (+15.3%). Karena SimpleRNN tidak memiliki mekanisme gating, kapasitas representasi yang lebih besar memberikan manfaat yang terasa. Varian yang lebih kecil pada H512 (std = 0.0076 vs 0.0134) mengindikasikan training yang lebih stabil. Waktu decoding meningkat sekitar $1.7\times$ dari H256 ke H512.



Gambar 8 Pengaruh hidden size terhadap rata-rata BLEU-4 (SimpleRNN dan LSTM)

4.2.4 Perbandingan Ukuran Hidden State: LSTM

Tabel 7 Rata-rata BLEU-4 LSTM per hidden size

Hidden	Mean BLEU-4	Std	Mean Decode Time (s)
256	0.1233	0.0105	13.1
512	0.1212	0.0119	20.2

Pada LSTM, perbedaan antara H256 dan H512 sangat kecil dan tidak signifikan: H256 bahkan sedikit unggul secara rata-rata (0.1233 vs 0.1212). LSTM dengan mekanisme gating sudah mampu memanfaatkan kapasitas representasi H256 secara efektif, sehingga

menambah hidden size ke 512 tidak memberikan keuntungan yang sepadan dengan penambahan parameter ($\approx 2\times$) dan waktu decoding ($\approx 1.5\times$). Untuk LSTM, H256 adalah pilihan yang lebih efisien.

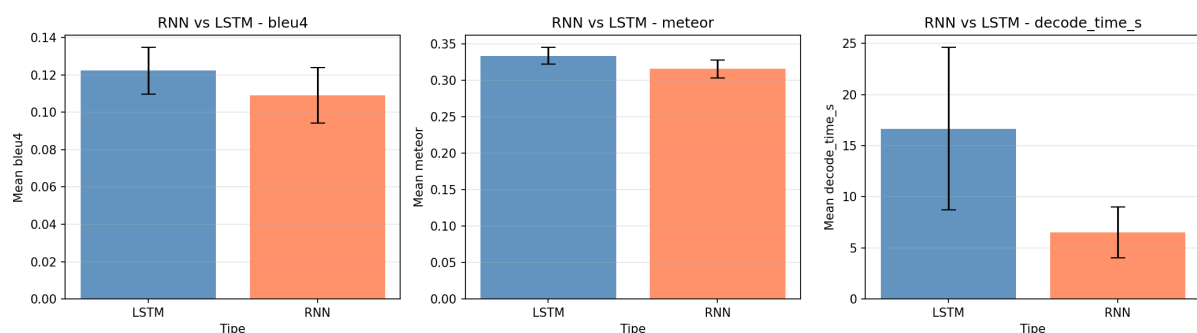
4.2.5 Perbandingan RNN vs LSTM

Tabel 8 Perbandingan rata-rata metrik SimpleRNN vs LSTM pada seluruh 6 variasi

Metrik	SimpleRNN	LSTM	Delta
BLEU-4	0.1090	0.1223	+12.2%
METEOR	0.3157	0.3335	+5.6%
Decode time (s)	6.51	16.66	+155.9%

LSTM unggul pada kedua metrik kualitas: BLEU-4 lebih tinggi 12.2% dan METEOR lebih tinggi 5.6%. Keunggulan ini berkaitan dengan kemampuan LSTM menangani dependensi jangka panjang. SimpleRNN memperbarui hidden state dengan satu operasi tanh dan rentan terhadap *vanishing gradient*, sehingga informasi gambar yang diinjeksikan di awal mudah terlupakan seiring bertambahnya panjang caption. LSTM mengatasi hal ini melalui cell state c_t yang membentuk jalur gradien lebih langsung sepanjang sequence, memungkinkan model mempertahankan konteks gambar dengan lebih baik.

Trade-off utama adalah kecepatan inference: LSTM membutuhkan waktu $2.6\times$ lebih lama dari SimpleRNN (16.66 s vs 6.51 s per 1.000 gambar), karena setiap cell LSTM menghitung empat gate pada setiap timestep.



Gambar 9 Perbandingan rata-rata BLEU-4, METEOR, dan decode time antara SimpleRNN dan LSTM

4.2.6 Perbandingan Keras vs From Scratch

Implementasi forward propagation NumPy diuji pada model terbaik masing-masing tipe (rnn_L1_H512 dan lstm_L1_H512) menggunakan 1.000 gambar test.

Tabel 9 Perbandingan Keras vs implementasi NumPy untuk model captioning terbaik

Model	Implementasi	BLEU-4	Waktu (s)	Speedup
rnn_L1_H512	Keras	0.1272	51.91	9.1×
	NumPy	0.1272	5.68	
lstm_L1_H512	Keras	0.1349	73.30	6.7×
	NumPy	0.1349	10.92	

Kedua implementasi menghasilkan BLEU-4 yang identik, memvalidasi kebenaran forward propagation NumPy dan transfer bobot dari Keras. Implementasi NumPy lebih cepat: 9.1× untuk SimpleRNN dan 6.7× untuk LSTM. Penyebab utamanya adalah strategi decoding: NumPy memperbarui state per satu timestep secara benar-benar autoregresif sehingga kompleksitasnya $O(T)$, sementara greedy decode pada Keras melakukan forward pass penuh (31 posisi) di setiap langkah decoding sehingga kompleksitasnya $O(T^2)$.

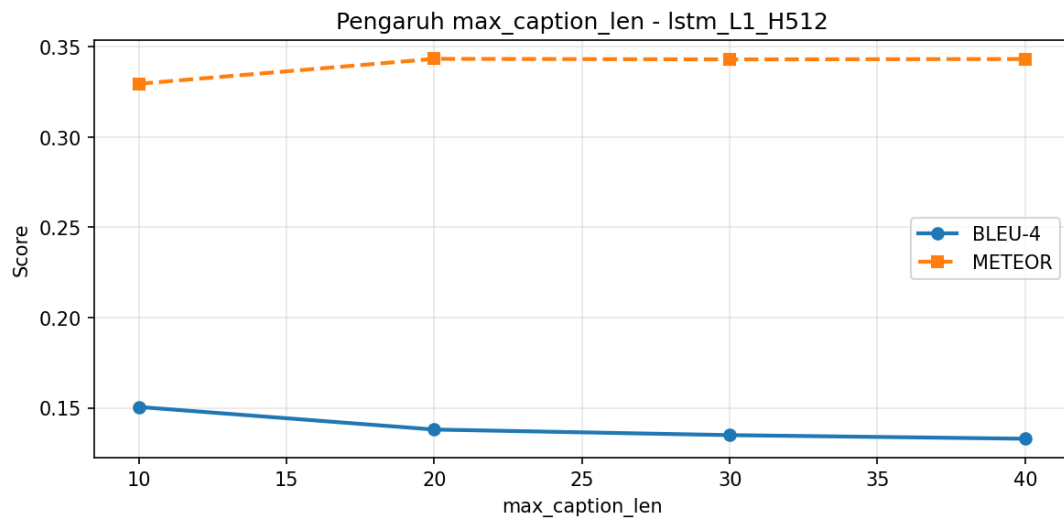
4.2.7 Pengaruh Panjang Maksimum Caption terhadap BLEU-4

Variasi `max_caption_len` diuji pada model terbaik (`lstm_L1_H512`).

Tabel 10 Pengaruh `max_caption_len` terhadap BLEU-4 dan METEOR pada `lstm_L1_H512`

<code>max_caption_len</code>	BLEU-4	METEOR	Decode Time (s)
10	0.1505	0.3295	8.22
20	0.1380	0.3433	10.05
30	0.1349	0.3430	10.16
40	0.1329	0.3432	10.36

BLEU-4 tertinggi dicapai pada `max_caption_len=10` (0.1505), lalu menurun seiring bertambahnya batas panjang. METEOR justru meningkat dari 10 ke 20 (0.3295 ke 0.3433) lalu relatif stabil hingga 40. Perbedaan perilaku kedua metrik ini mencerminkan karakteristiknya: BLEU-4 mengukur presisi n-gram, sehingga caption pendek dengan sedikit token cenderung memiliki presisi lebih tinggi karena lebih sedikit peluang menghasilkan n-gram yang salah. METEOR yang berbasis recall unigram dan synonym lebih sensitif terhadap kelengkapan konten, sehingga caption yang terlalu pendek kehilangan detail dan skornya lebih rendah. Waktu decoding meningkat perlahan dari 8.22 s ke 10.36 s seiring bertambahnya batas, karena model perlu memproses lebih banyak timestep sebelum menghasilkan token `<end>`.



Gambar 10 Pengaruh max_caption_len terhadap BLEU-4 dan METEOR pada lstm_L1_H512

5 Kesimpulan

Klasifikasi Gambar dengan CNN

Dari 16 variasi model Conv2D yang diuji, model terbaik berdasarkan validation F1 adalah conv_L4_F64-128-256-512_K3_Pa dengan test macro F1 = 0.8723. Berikut beberapa temuan utama dari eksperimen CNN:

- **Jumlah layer:** Model 4-layer secara konsisten mengungguli model 3-layer (rata-rata test F1 0.8685 vs 0.8524). Layer yang lebih banyak memungkinkan ekstraksi fitur yang lebih hierarkis dan abstrak.
- **Jumlah filter:** Perbedaan antara schedule filter kecil dan besar hampir tidak ada (0.8604 vs 0.8606 rata-rata F1), sehingga filter yang lebih besar tidak memberikan keuntungan yang sepadan dengan biaya komputasi tambahan.
- **Ukuran kernel:** Kernel 5×5 sedikit lebih unggul dari 3×3 secara rata-rata (0.8654 vs 0.8556), namun model terbaik keseluruhan tetap menggunakan 3×3 .
- **Jenis pooling:** Max pooling dan average pooling memberikan performa yang hampir identik (selisih rata-rata F1 kurang dari 0.0005).
- **Shared vs non-shared parameter:** LocallyConnected2D gagal konvergen dengan test F1 = 0.0496, jauh di bawah Conv2D (0.8723), meskipun memiliki 34 kali lebih banyak parameter (235 juta vs 6,8 juta). Ini membuktikan bahwa *weight sharing* pada Conv2D merupakan *inductive bias* yang sangat penting untuk data visual.

Implementasi NumPy menghasilkan prediksi yang identik dengan Keras pada seluruh 3.000 sampel test (agreement 100%), memvalidasi kebenaran forward propagation. Waktu inferensi NumPy sekitar 37 kali lebih lambat karena tidak memanfaatkan akselerasi GPU.

Image Captioning dengan RNN dan LSTM

Dari 12 variasi model decoder yang diuji, model terbaik adalah lstm_L1_H512 dengan BLEU-4 = 0.1349 dan METEOR = 0.3430. Berikut beberapa temuan utama dari eksperimen RNN dan LSTM:

- **Jumlah recurrent layer:** Model satu layer menghasilkan performa terbaik pada kedua arsitektur. BLEU-4 menurun sekitar 19% dari L1 ke L3 untuk keduanya, kemungkinan karena model yang lebih dalam sulit dioptimalkan pada dataset berukuran 6.000 gambar.

- **Hidden size:** Pada SimpleRNN, H512 lebih unggul dari H256 (+15.3% BLEU-4) karena kapasitas representasi yang lebih besar. Pada LSTM, perbedaannya tidak signifikan sehingga H256 lebih efisien.
- **SimpleRNN vs LSTM:** LSTM unggul pada semua konfigurasi dengan selisih rata-rata BLEU-4 +12.2% dan METEOR +5.6%. Mekanisme gating LSTM memungkinkan aliran gradien yang lebih baik sehingga konteks gambar yang diinjeksikan di awal dapat dipertahankan lebih lama. Trade-off utamanya adalah waktu inference yang 2.6 kali lebih lambat.
- **Panjang maksimum caption:** BLEU-4 tertinggi dicapai pada `max_caption_len=10` (0.1505) karena precision n-gram lebih tinggi pada caption pendek, sementara METEOR lebih stabil setelah `max_caption_len=20` karena berbasis recall unigram.

Implementasi NumPy untuk captioning juga menghasilkan BLEU-4 identik dengan Keras, dengan kecepatan 9.1 kali lebih cepat untuk SimpleRNN dan 6.7 kali untuk LSTM. Keunggulan kecepatan ini berasal dari strategi decoding autoregresif $O(T)$ pada NumPy, dibandingkan dengan $O(T^2)$ pada greedy decode di atas model Keras.

6 Pembagian Tugas

NIM	Nama	Kontribusi
13523037	Buege Mahara Putra	Seluruhnya

Referensi

1. Keras Team. *The Sequential model*. Keras Documentation.
https://keras.io/guides/sequential_model/
2. Keras Team. *The Functional API*. Keras Documentation.
https://keras.io/guides/functional_api/
3. Keras Team. *Training & evaluation with the built-in methods*. Keras Documentation.
https://keras.io/guides/training_with_built_in_methods/
4. Keras Team. *Save, serialize, and export models*. Keras Documentation.
https://keras.io/guides/serialization_and_saving/
5. TensorFlow Team. *tf.data: Build TensorFlow input pipelines*. TensorFlow Documentation.
<https://www.tensorflow.org/guide/data>
6. NumPy Team. *Broadcasting*. NumPy Documentation.
<https://numpy.org/doc/stable/user/basics.broadcasting.html>
7. NumPy Team. *numpy.einsum*. NumPy Documentation.
<https://numpy.org/doc/stable/reference/generated/numpy.einsum.html>
8. Zhang, A., et al. *Convolutional Neural Networks*. Dive into Deep Learning (d2l.ai).
https://d2l.ai/chapter_convolutional-neural-networks/
9. Zhang, A., et al. *Recurrent Neural Networks — Implementation from Scratch*. Dive into Deep Learning (d2l.ai).
https://d2l.ai/chapter_recurrent-neural-networks/rnn-scratch.html
10. Zhang, A., et al. *Long Short-Term Memory (LSTM)*. Dive into Deep Learning (d2l.ai).
https://d2l.ai/chapter_recurrent-modern/lstm.html
11. Zhang, A., et al. *Sequence to Sequence Learning*. Dive into Deep Learning (d2l.ai).
https://d2l.ai/chapter_recurrent-modern/seq2seq.html
12. Keras Team. *Image Captioning*. Keras Examples.
https://keras.io/examples/vision/image_captioning/

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/~~kami~~:

Nama/NIM : Buege Mahara Putra (13523037)

Fakultas/Sekolah : STEI

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2: Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa saya/~~kami~~* menggunakan/~~tidak~~ menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	
Lainnya, Jelaskan:		

Saya/~~kami~~* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026



Buege Mahara Putra
13523037